FindIt — Design Documentation

Architecture, MVC flow, and persistence design

1. System Overview

FindIt is described as a Spring Boot web application. The following design reflects the discussed layered structure; verify class names and layers against the source code.

2. Architecture Components

View: Thymeleaf HTML templates render the user interface.
Controller: HomeController handles web requests and selects responses.
Service: ItemService contains business logic, if present.
Repository: ItemRepository provides data-access methods, if present.
Entity: Item represents persistent item data, if mapped with JPA.
Persistence: Spring Data JPA and Hibernate can handle object-relational persistence.

3. Request Flow

1. A user requests a page or submits a form in the browser.
2. The embedded web server (commonly Tomcat) receives the HTTP request.
3. Spring MVC's DispatcherServlet routes the request to a matching controller handler.
4. The controller processes the request and may call the service layer.
5. The service may use the repository to read or write database records.
6. The controller returns a view name or other response.
7. Thymeleaf renders the HTML template when a view is returned.

4. MVC Design

Model: item data and related application state.
View: HTML templates rendered using Thymeleaf.
Controller: request handling and coordination between the view and application logic.

5. Database Design

Document the actual Item entity fields, Java types, primary key, constraints, and table mapping from the source code. Do not assume specific fields until verified. If MySQL is used, record the database name and relevant relationships.

6. Design Considerations

• Keep request handling, business logic, and database access separated where the project architecture supports it.
• Validate form input before saving data.
• Keep credentials outside public source control.
• Show understandable error messages without exposing sensitive technical details.

7. Items to Confirm Before Submission

Confirm the real package structure, controller mappings, entity fields, database type, implemented features, and whether a separate service layer exists. Update this document so it describes the actual application rather than a planned design.